

Modelling and Simulation Lab

General Instructions

- Maintain a lab record with date, aim, MATLAB code, Simulink model screenshots, outputs, observations, and conclusion.
- Save all .m, .mlx, and .slx files with proper names and version numbers.
- Use comments in code for clarity and mention assumptions for each case study.
- Validate numerical outputs, units, plots, and solver settings before submission.
- Record input data source, parameter values, and interpretation of results.

Software and Resources

Typical software used in this laboratory includes MATLAB, Simulink, Neural Network / Deep Learning related toolboxes, Statistics and Machine Learning tools, and domain-specific add-ons where available. MATLAB supports matrix-based technical computing, while Simulink provides graphical block-diagram modeling for dynamic systems and model-based design.

Experiment 1

Title- Introduction to programming with MATLAB.

Tools/ Software Used

- MATLAB

Expected Outcomes

- Familiarity with MATLAB environment including command window, editor, workspace, and current folder.
- Ability to write and execute basic scripts using variables, arrays, operators, matrices, vectors, and built-in functions.
- Ability to generate 2D plots and interpret results visually.

Detailed Procedure

1. Open MATLAB and study the command window, editor, workspace, and help browser.
2. Create a new script file and save it in the current working directory.
3. Write basic commands using scalar variables, arithmetic operators, relational operators, and logical operators.
4. Create vectors and matrices using direct entry, `linspace`, `zeros`, `ones`, and `rand`.
5. Perform matrix operations such as transpose, determinant, inverse, addition, multiplication, and element-wise operations.
6. Use built-in functions such as `sin`, `cos`, `tan`, `sqrt`, and `exp` for numerical computation.
7. Plot simple engineering functions such as sine, cosine, ramp, step, and circle using `plot`, `xlabel`, `ylabel`, `title`, and `subplot`.
8. Execute the script, observe command window and figure output, and correct syntax or logic errors if found.

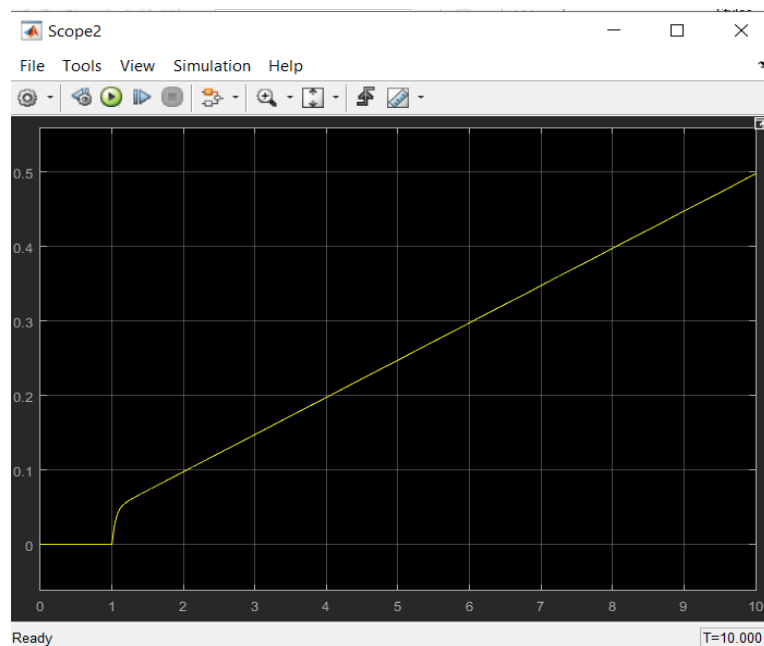
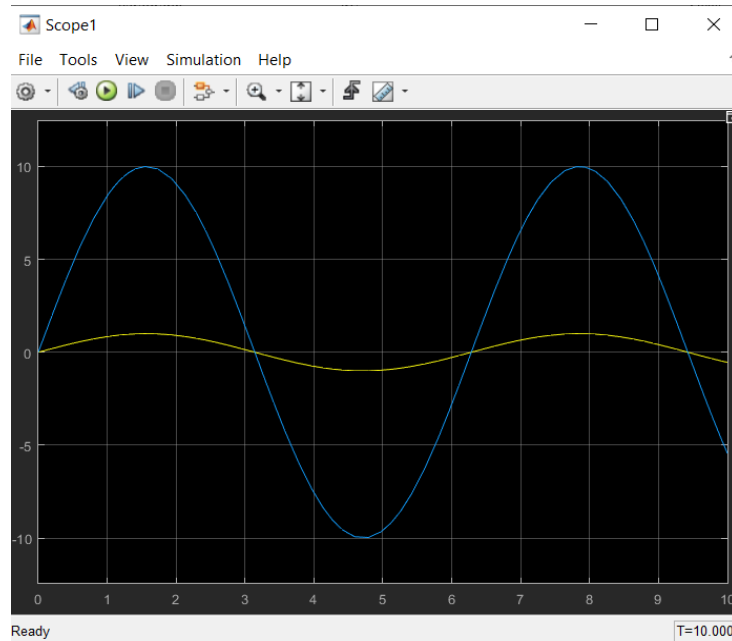
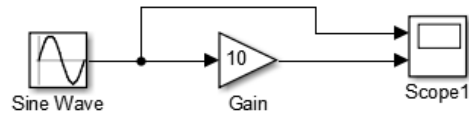
Summary

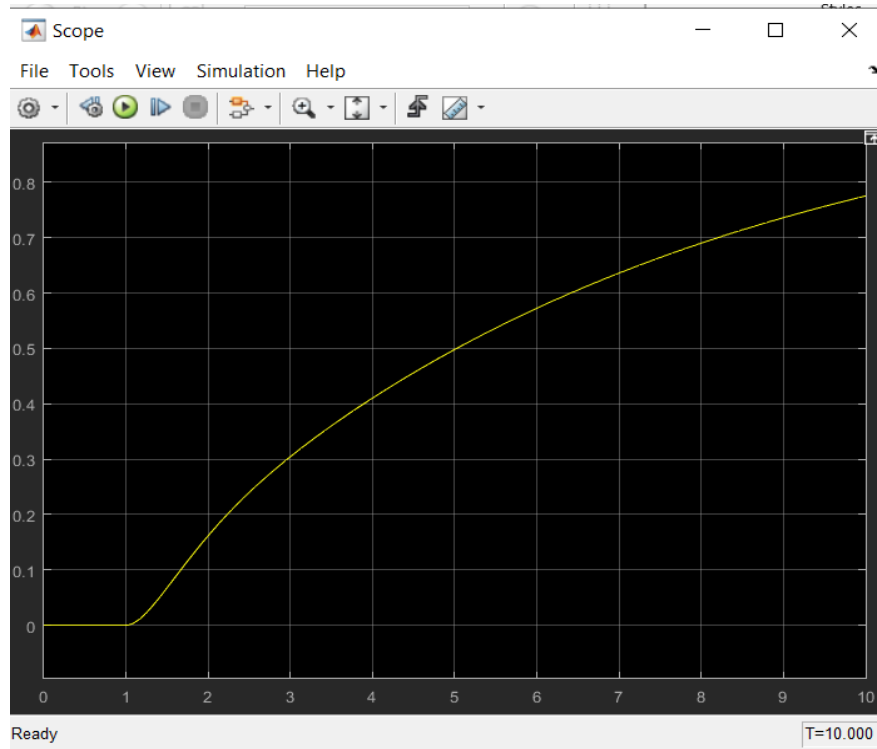
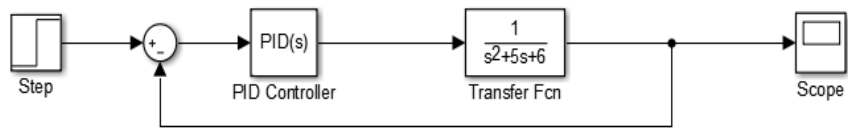
This experiment builds the foundation for all later MATLAB work by introducing scripting, matrix operations, and plotting. Since MATLAB is centered on array-based computing and visualization, these skills are essential for engineering analysis and simulation.

Experiment 2

Title: Case study on Simulink in MATLAB for Engineering Problems

Aim: To build following simple models in Simulink and observe simulated results.





Experiment 3

Title: Case study on Use of Neural Network in MATLAB for Engineering Problem

A **Neural Network** is a computational model inspired by the human brain, used for:

- Pattern recognition
- Prediction
- System modeling

In engineering, neural networks are widely used for:

- Sensor calibration
- Fault detection
- System identification

MATLAB provides built-in support through its neural network capabilities within **MATLAB**.

Problem Statement

Predict System Output from Input Data

We will model a simple engineering system:

Predict output y for a nonlinear system:

$$y=x^2$$

This represents:

- Nonlinear sensor behaviour
- Robot actuator response

Objectives

- Train a simple neural network (Input layer, hidden layer, output layer)
- Approximate a nonlinear function
- Compare actual vs predicted output

MATLAB code:

```
% Define input ,output data
```

```
x = -10:1:10;    % Input
```

```
y = x.^2;        % Output (target)
```

```
% create NN with 10 neurons in hidden layer
```

```
net = feedforwardnet(10);
```

```
net = train(net, x, y); %Train the neural network
```

```
y_pred = net(x); %Test the network
```

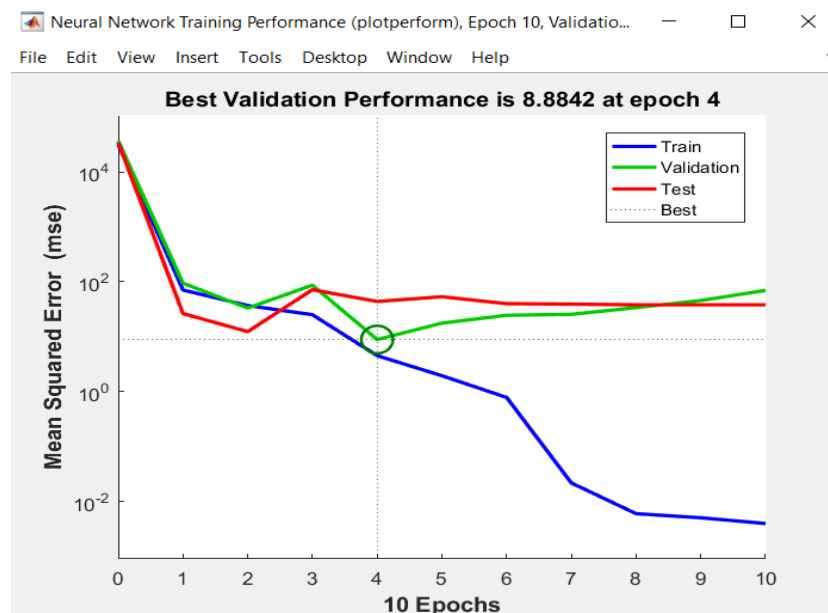
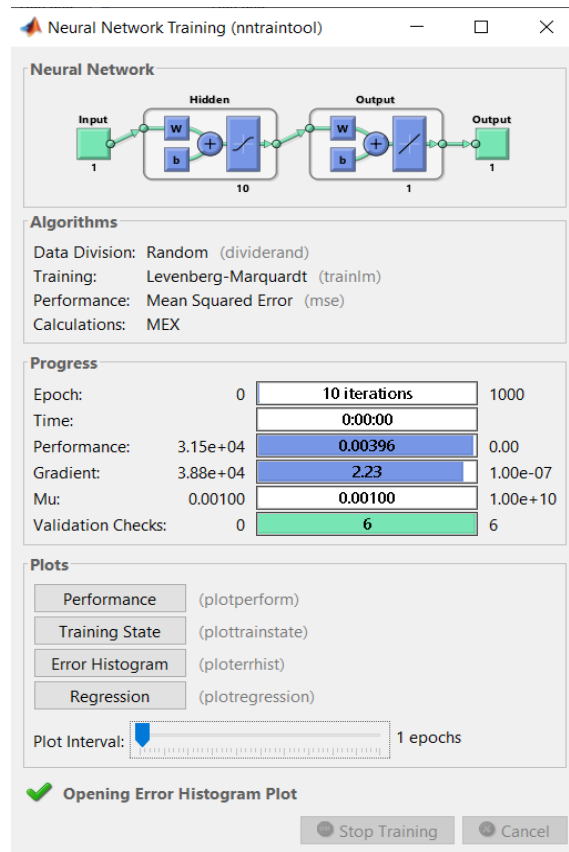
```
%Plot results
```

```
plot(x, y, 'b', x, y_pred, 'r--');
```

```
legend('Actual Output', 'Predicted Output');
```

```
title('Neural Network Approximation');
```

```
grid on;
```



feedforwardnet(10)

- Creates a neural network
- 10 neurons in hidden layer
- Suitable for simple nonlinear problems

train()

- Adjusts weights using training data
- Uses backpropagation algorithm

net(x)

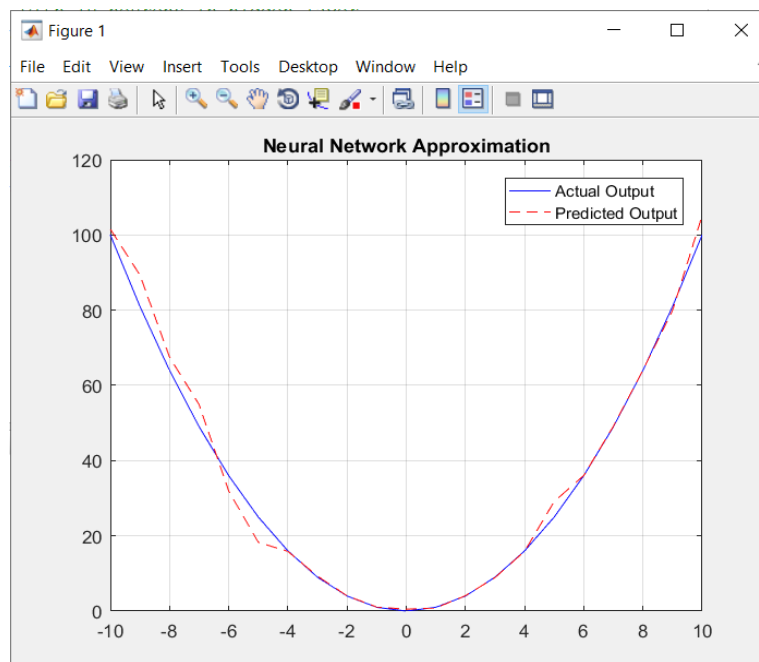
- Produces predicted output

Sample Output (Expected)

- Blue line → Actual $y=x^2$
- Red dashed line → Neural network prediction

Observation

- Prediction closely follows actual curve
- Small errors may occur at extremes



This simple model demonstrates:

Function Approximation

Used in:

- Robot dynamics modeling
- Sensor nonlinear calibration

Learning from Data

No need for exact mathematical model

Real Applications

- Predict motor torque
- Estimate battery behaviour
- Fault prediction in machines

Students can modify:

- Number of neurons
- Input range
- Noise addition

Example:

```
y = x.^2 + rand(1,length(x)); % Add noise
```

Advantages of Neural Networks

- Handles nonlinear systems
- Learns from data
- Flexible and adaptive

Limitations

- Requires training data
- Can overfit
- Less interpretable than equations

To convert this to a Simulink model automatically, use:

```
gensim(net)
```

This automatically creates a **Simulink model** with your trained neural network.

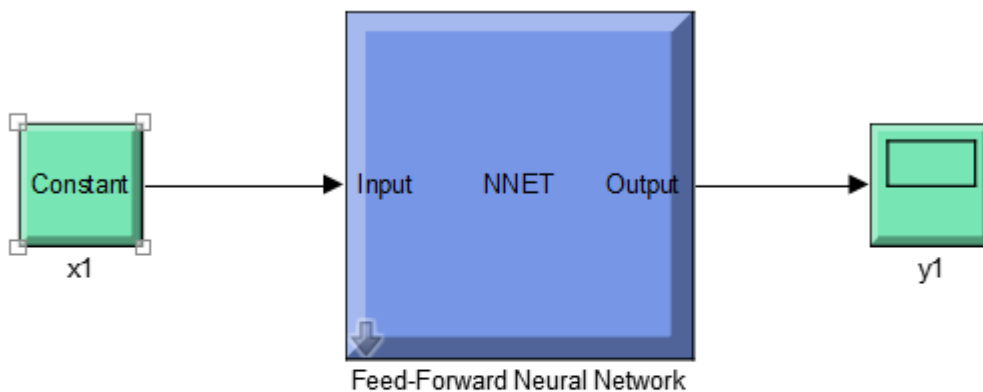
Simulink Model Structure

The generated model will include:

- **Input block** (your x values)
- **Neural Network block**
- **Output block**

You can modify it by adding:

- Scope (to visualize output)
- Compare with actual x^2



Conclusion:

This case study shows how a simple neural network in MATLAB can model nonlinear engineering systems.

Experiment 4

Title: Use of MATLAB for DC Motor Speed Control

This example demonstrates System modelling, Controller design (PID) ,Simulation and Performance analysis

```
clc;
clear;
close all;
%System Parameters (DC Motor)
J = 0.01; % Moment of inertia
b = 0.1; % Damping coefficient
K = 0.01; % Motor constant
R = 1; % Resistance
L = 0.5; % Inductance
% Transfer Function of DC Motor
% Theta(s)/V(s)
num = K;
den = [J*L (J*R + b*L) (b*R + K^2)];
plant = tf(num, den);
disp('DC Motor Transfer Function:');
plant
% Open Loop Response
figure;
step(plant);
title('Open Loop Step Response');
grid on;
% PID Controller Design
Kp = 100;
Ki = 200;
Kd = 10;
controller = pid(Kp, Ki, Kd);
% Closed Loop System
system_cl = feedback(controller * plant, 1);
% Closed Loop Response
figure;
step(system_cl);
title('Closed Loop Step Response with PID Controller');
grid on;
% Performance Metrics
info = stepinfo(system_cl);
disp('Performance Metrics:');
disp(info);
%. Disturbance Rejection Test
t = 0:0.01:5;
```

```

u = ones(size(t)); % Step input
disturbance = 0.2*sin(2*t); % disturbance signal
[y, t_out] = lsim(system_cl, u + disturbance, t);
figure;
plot(t_out, y, 'b', 'LineWidth', 2);
hold on;
plot(t, u, 'r--');
title('System Response with Disturbance');
legend('Output', 'Reference');
grid on;

```

Sample output:

DC Motor Transfer Function:

plant =

0.01

 $0.005 s^2 + 0.06 s + 0.1001$

Continuous-time transfer function.

Performance Metrics:

RiseTime: 0.1324

SettlingTime: 0.2570

SettlingMin: 0.9010

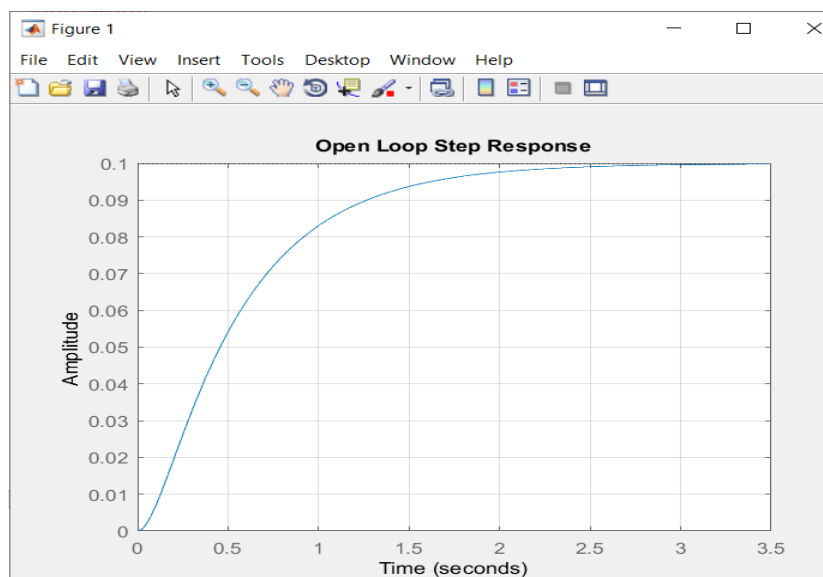
SettlingMax: 1.0103

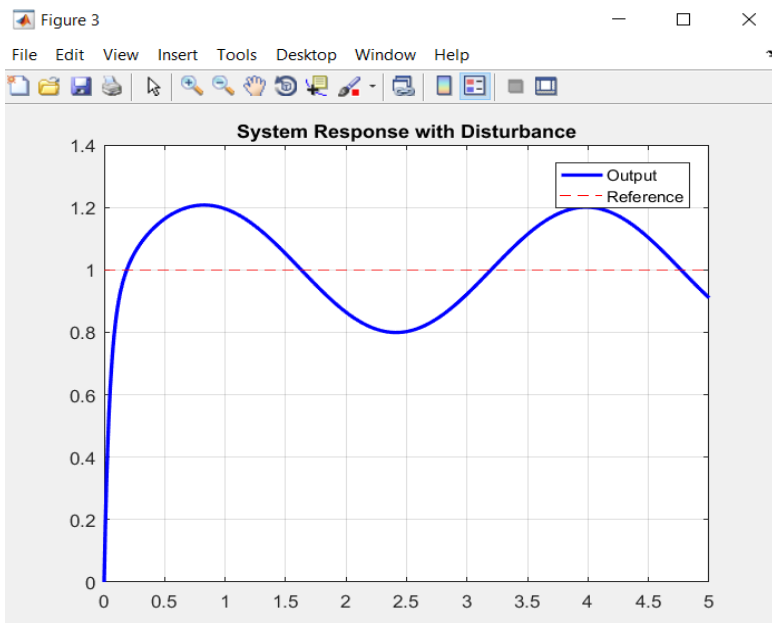
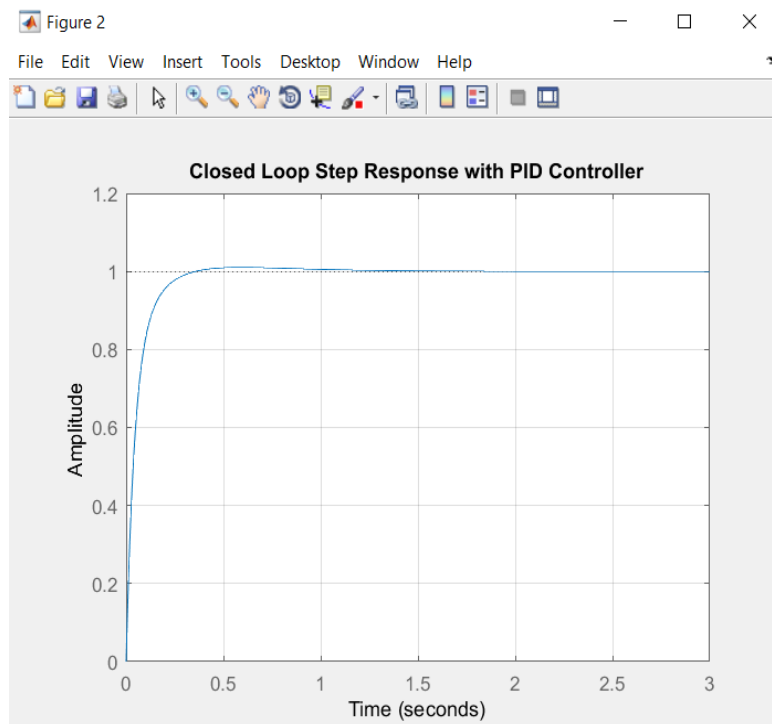
Overshoot: 1.0281

Undershoot: 0

Peak: 1.0103

PeakTime: 0.5932





Experiment 5

Title: Case study on an automated manufacturing system, where a sensor records temperature data from a machine over time.

The goal is to:

- Analyze input data
- Detect anomalies
- Extract useful insights for automation decisions

Input Dataset (Example)

Assume temperature readings (°C) collected every second:

Objectives of Analysis

- Visualize input behavior
- Compute statistical properties
- Detect abnormal spikes
- Smooth noisy data
- Support automation decisions

Complete Matlab code:

```
clc;
clear;
close all;

time = (0:1:50)';
temperature = [30 31 29 32 33 35 36 37 38 39 ...
              40 42 41 43 45 47 50 52 55 60 ...
              65 70 75 80 85 90 95 100 105 110 ...
              108 107 106 105 104 103 102 101 ...
              100 99 98 97 96 95 94 93 92 91 90 91 76]';

figure;
plot(time, temperature, 'b-o');
title('Temperature vs Time');
xlabel('Time (s)');
ylabel('Temperature (°C)');
grid on;
mean_temp = mean(temperature);
max_temp = max(temperature);
min_temp = min(temperature);
std_temp = std(temperature);

fprintf('Mean: %.2f\nMax: %.2f\nMin: %.2f\nStd Dev: %.2f\n', ...
        mean_temp, max_temp, min_temp, std_temp);
smooth_temp = movmean(temperature, 5);

figure;
plot(time, temperature, 'b');
hold on;
plot(time, smooth_temp, 'r', 'LineWidth', 2);
legend('Original Data', 'Smoothed Data');
title('Noise Reduction using Moving Average');
```

```

grid on;
threshold = 100; % Safety limit
anomalies = temperature > threshold;

```

```

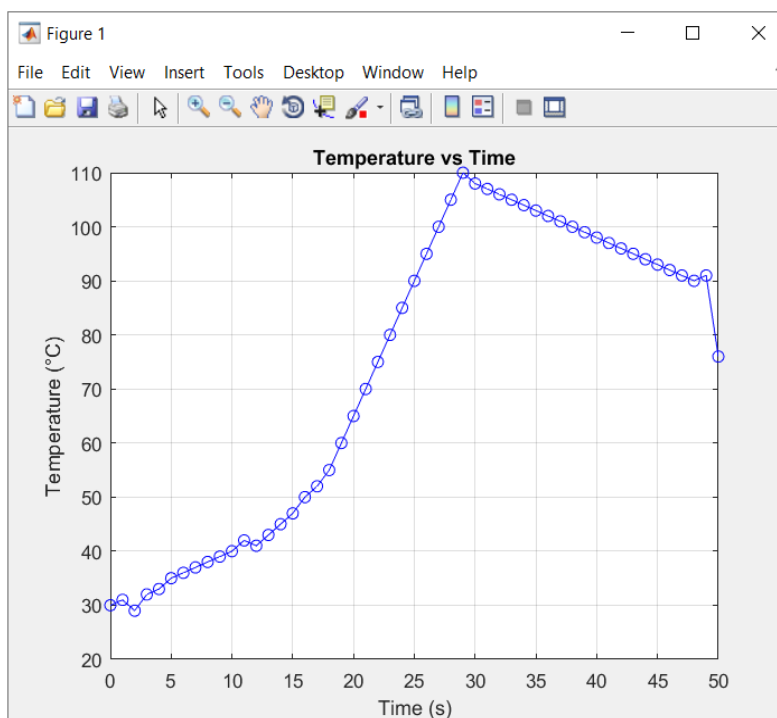
figure;
plot(time, temperature, 'b');
hold on;
plot(time(anomalies), temperature(anomalies), 'ro', 'MarkerSize', 8);
title('Anomaly Detection');
legend('Normal Data', 'Anomalies');
grid on;
p = polyfit(time, temperature, 2); % Quadratic fit
trend = polyval(p, time);

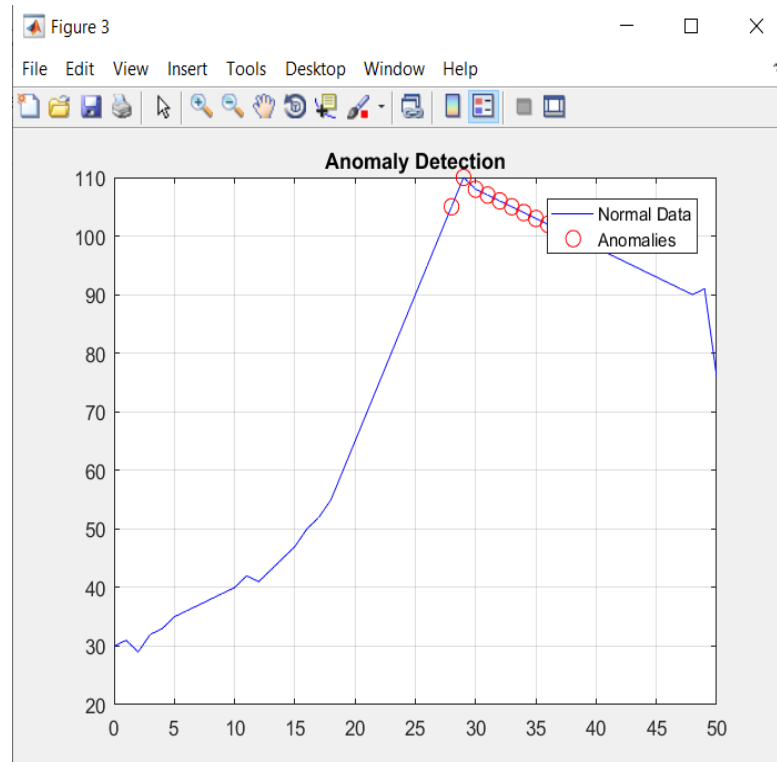
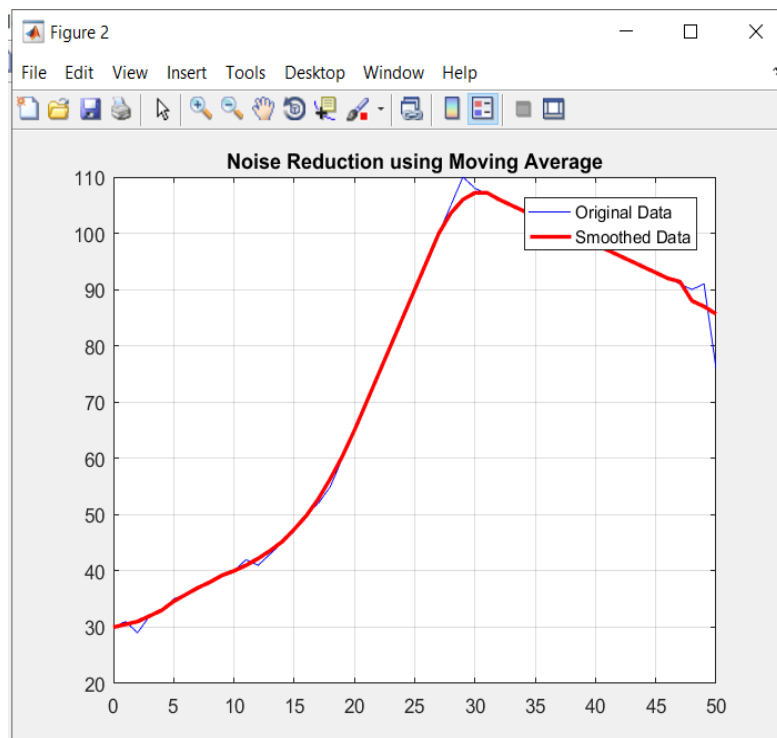
```

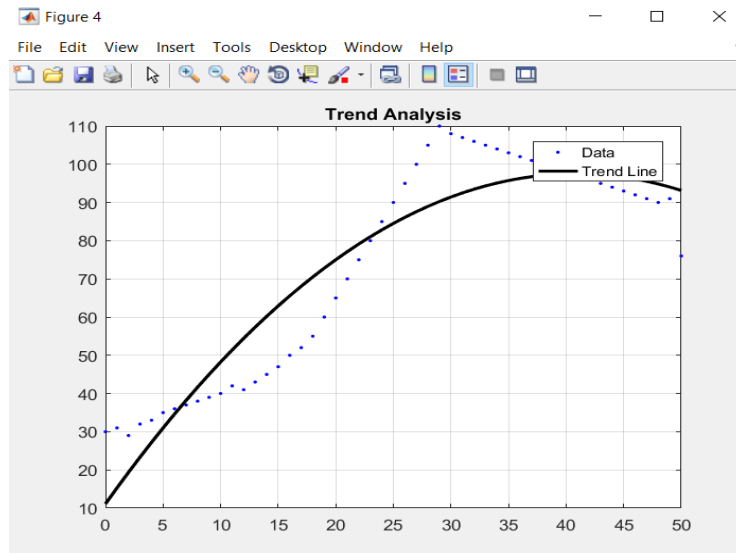
```

figure;
plot(time, temperature, 'b. ');
hold on;
plot(time, trend, 'k', 'LineWidth', 2);
title('Trend Analysis');
legend('Data', 'Trend Line');
grid on;

```







Output:

Mean: 73.29

Max: 110.00

Min: 29.00

Std Dev: 28.33

Experiment 6

Title: Case Study on Input Data Analysis - Machining Process Monitoring Data

Tools

- MATLAB command window and editor
- Data import functions (readtable, csvread, xlsread)
- Statistical functions (mean, std, corr, histogram)
- Plotting functions (boxplot, scatter, heatmap)

Software Used

- MATLAB (R2024a or later recommended)
- Statistics and Machine Learning Toolbox (optional)

Expected Outcomes

- Complete statistical characterization of engineering input data
- Identification of missing values, outliers, and data quality issues
- Visualization of data distributions, correlations, and trends
- Preprocessed dataset ready for modeling/simulation
- Data quality report with recommendations

Detailed Procedure

1. Dataset Overview

Sample machining dataset (1000 data points):

Columns: Time(s), Spindle_RPM, Feed_rate(mm/min),
Current(A), Vibration(μ m), Surface_Ra(μ m)

2. Data Import and Initial Inspection

```
data = readtable('machining_data.csv');  
summary(data) % Dimensions, variable types, missing values
```

3. Missing Value Analysis

```
missing = sum(ismissing(data))/height(data)*100;  
% Strategy: interpolation for time-series, mean for steady-state  
data.Feed_rate = fillmissing(data.Feed_rate, 'linear');
```

4. Descriptive Statistics

```
stats = groupsummary(data, 'all', ...  
    {'mean','std','min','max','iqr'}, ...  
    {'Spindle_RPM','Current','Vibration'});
```

5. Data Visualization Suite


```

% Univariate analysis
subplot(2,3,1); histogram(data.Spindle_RPM); title('RPM Distribution');
subplot(2,3,2); boxplot(data.Current); title('Current');
subplot(2,3,3); histogram(data.Surface_Ra); title('Ra Distribution');

% Bivariate analysis
subplot(2,3,4); scatter(data.Spindle_RPM, data.Surface_Ra);
subplot(2,3,5); scatter(data.Feed_rate, data.Vibration);

% Correlation matrix
subplot(2,3,6); heatmap(corr(data{:,3:end}), ...
    XLabel=data.Properties.VariableNames(3:end));

```

6. Outlier Detection

```

% IQR method
Q1 = quantile(data.Vibration, 0.25);
Q3 = quantile(data.Vibration, 0.75);
IQR = Q3 - Q1;
outliers = data.Vibration < (Q1-1.5*IQR) | ...
    data.Vibration > (Q3+1.5*IQR);
fprintf('Outliers: %.1f%%\n', mean(outliers)*100);

```

7. Data Quality Metrics

```

% Skewness and kurtosis
skewness_metrics = skewness(data{:,3:end});
kurtosis_metrics = kurtosis(data{:,3:end});

% Autocorrelation for time-series
autocorr(data.Current, 20);

```

8. Preprocessing Pipeline

```

% Normalization options
data_norm.Z_RPM = zscore(data.Spindle_RPM);
data_norm.MinMax_Current = (data.Current - min(data.Current)) / ...
    (max(data.Current) - min(data.Current));

% Trend removal (for stationary analysis)
[detrended, trend] = detrend(data.Vibration);

```

Sample Analysis Results

Variable	Mean	Std	Range	Missing (%)	Outliers (%)	Skewness
RPM	1520	180	1000-2000	0.0	2.1	-0.12
Current	8.45	1.2	6.2-11.0	0.5	3.8	0.45
Vibration	12.3	3.1	5.0-22.0	1.2	4.2	0.78
Surface (Ra)	2.15	0.45	1.2-3.8	0.0	2.9	0.62

Key Diagnostic Plots

1. Distribution plots (histograms + boxplots)
2. Scatter plot matrix (multivariate relationships)
3. Correlation heatmap (feature selection guidance)
4. Time-series plots (trends, seasonality)
5. Residual plots (after detrending)
6. Autocorrelation plots (time dependencies)

Data Quality Assessment Checklist

Issue	Detected	Action Taken	Impact
Missing values	0.5-1.2%	Linear interpolation	Minimal
Outliers	2-5%	Flagged, not removed	Investigate tool wear
Non-normality	Vibration, Ra	Log transform option	Consider for modeling
High correlation	RPM-Current (0.82)	Feature selection	Multicollinearity risk
Time dependencies	Current (lag=2)	ARIMA preprocessing	Required for prediction

Summary

This experiment establishes systematic data analysis workflow essential before any engineering modeling or machine learning application. The machining dataset analysis reveals:

1. Data quality is generally good (low missing values, reasonable ranges)
2. Vibration and Surface Roughness show non-normal distributions requiring transformation
3. Strong RPM-Current correlation suggests potential multicollinearity
4. Moderate outliers likely represent genuine process variation (tool wear, faults)

Experiment 7

Title: Case Study on simulation of manufacturing system

Basic programs:

1. Hello World (Basic Output)

```
disp('Hello, MATLAB!');
```

Basic Arithmetic Operations

```
a = 10;
```

```
b = 5;
```

```
sum = a + b
```

```
difference = a - b
```

```
product = a * b
```

```
division = a / b
```

User Input

```
name = input('Enter your name: ', 's');
```

```
age = input('Enter your age: ');
```

```
fprintf('Hello %s, you are %d years old.\n', name, age);
```

For Loop (Print Numbers 1 to 10)

```
for i = 1:10
```

```
    disp(i);
```

```
end
```

While Loop

```
i = 1;
```

```
while i <= 5
```

```
    disp(i);
```

```
    i = i + 1;
```

```
end
```

If-Else Condition

```
num = input('Enter a number: ');
```

```
if num > 0
```

```
    disp('Positive number');
```

```
elseif num < 0
```

```
    disp('Negative number');
```

```
else
    disp('Zero');
end
```

Array Operations

```
A = [1 2 3 4 5];

sum_A = sum(A)
max_A = max(A)
min_A = min(A)
```

Simple Plot

```
x = 0:0.1:10;
y = x.^2;

plot(x, y);
title('y = x^2');
xlabel('x');
ylabel('y');
grid on;
```

Matrix Operations

```
A = [1 2; 3 4];
B = [5 6; 7 8];

C = A + B
D = A * B
transpose_A = A'
```

Simple Function

Create a file named square.m:

```
function y = square(x)
    y = x^2;
end
```

Use it:

```
result = square(5)
```

Random Numbers

```
r = rand(1,5); % 5 random numbers between 0 and 1
disp(r);
```

Mini Example (Even or Odd)

```
num = input('Enter a number: ');
```

```
if mod(num,2) == 0
```

```
    disp('Even number');
```

```
else
```

```
    disp('Odd number');
```

```
end
```